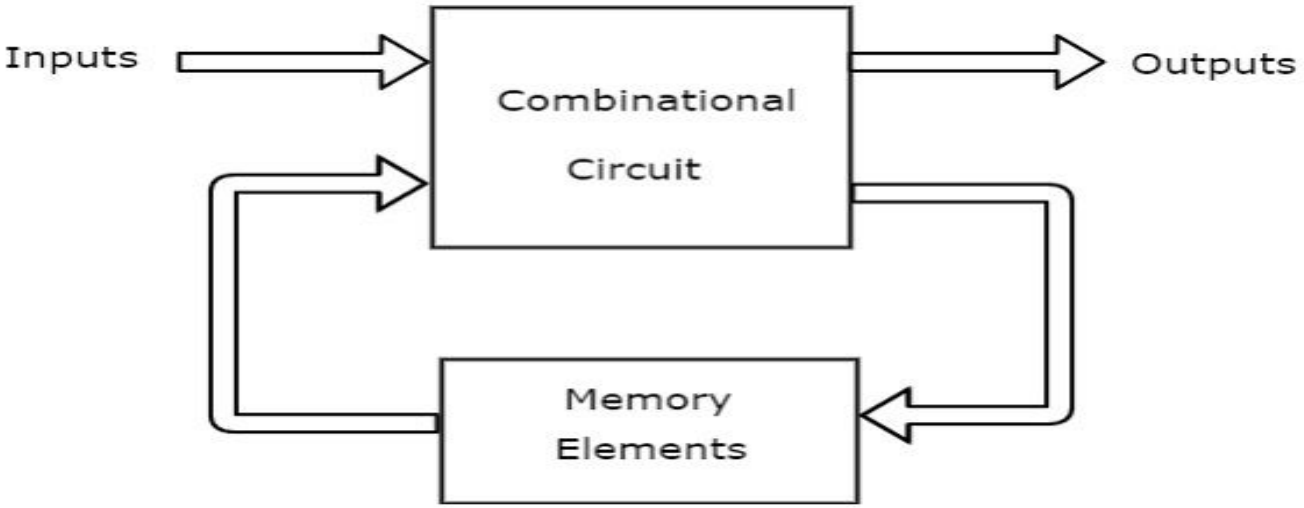
Sequential Circuits

- Sequential circuit contains a set of inputs and outputs
- The outputs of sequential circuit depends not only on the combination of present inputs but also on the previous outputs.
- Previous output is nothing but the present state. Therefore, sequential circuits contain combinational circuits along with memory storage elements.
- Some sequential circuits may not contain combinational circuits, but only memory elements.



- The differences between combinational circuits and sequential circuits.

	Combinational Circuits	Sequential Circuits
1.	The outputs of the combinational circuit depend only on the present inputs.	The outputs of the sequential circuits depend on both present inputs and present state(previous output).
2.	The feedback path is not present in the combinational circuit.	The feedback path is present in the sequential circuits.
3.	In combinational circuits, memory elements are not required.	In the sequential circuit, memory elements play an important role and require.
4.	The clock signal is not required for combinational circuits.	The clock signal is required for sequential circuits.
5.	The combinational circuit is simple to design.	It is not simple to design a sequential circuit.

Types of Sequential Circuits

Asynchronous sequential circuits

- The Asynchronous sequential circuits do not use the clock signals.
- The asynchronous circuit is operated through the pulses. So, the changes in the input can change the state of the circuit.
- The internal state is changed when the input variable is changed.
- The asynchronous sequential circuit is similar to the combinational circuits with feedback.

Synchronous sequential circuits

- In synchronous sequential circuits, synchronization of the memory element's state is done by the clock signal.
- The output is stored in either flip-flops or latches(memory devices).
- The synchronization of the outputs is done with either only negative edges of the clock signal or only positive edges.

Clock Signal and Triggering

Clock signal

- Clock signal is a periodic signal and its ON time and OFF time need not be the same.
- We can represent the clock signal as a square wave, when both its ON time and OFF time are same.

Flip-flop

- A flip-flop is a sequential digital electronic circuit having two stable states that can be used to store one bit of binary data.
- Flip-flops are the fundamental building blocks of all memory devices.

Types of Flip-Flops

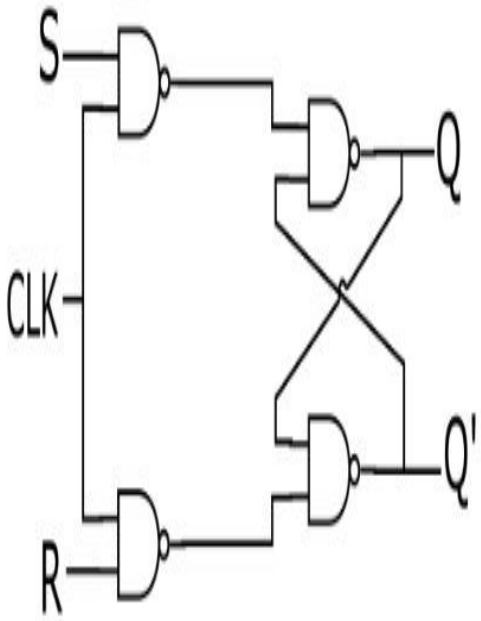
- S-R Flip-Flop
- J-K Flip-Flop
- D Flip-Flop
- T Flip-Flop

S-R Flip-Flop

- This is the simplest flip-flop circuit. It has a set input (S) and a reset input (R).
- In this circuit, when S is set as active, the output Q will be high and the Q’ will be low.
- If R is set to active, then the output Q is low and the Q' is high.
- Once the outputs are established, the results of the circuit are maintained until S or R get changed, or the power is turned off.

Truth Table of S-R Flip-Flop

S	R	Q	State
0	0	0	No Change
0	1	0	Reset
1	0	1	Set
1	1	X	



Characteristics Table of S-R Flip-Flop

S	R	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X

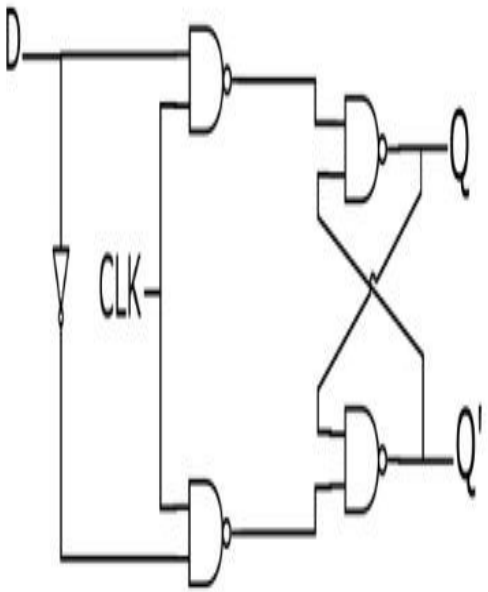
Characteristics Equation of S-R Flip-Flop

$$Q(t+1) = S + R' Q(t)$$

D Flip Flop

- In SR NAND Gate Bistable circuit, the undefined input condition of SET = "0" and RESET = "0" is forbidden.
- We connect the inverter between the Set and Reset inputs for producing another type of flip flop circuit called D flip flop, Delay flip flop, D-type Bistable, D-type flip flop.
- The D flip flop is the most important flip flop from other clocked types.
- It ensures that at the same time, both the inputs, i.e., S and R, are never equal to 1.

- The Delay flip-flop is designed using a gated SR flip-flop with an inverter connected between the inputs allowing for a single input D(Data).
- This single data input, which is labeled as "D" used in place of the "Set" input and for the complementary "Reset" input, the inverter is used. The lock and circuit diagram are shown below.
- When the value of Clock pulse is “1,” the D input is transferred to the flip flop.
- When clock pulse is high the flip-flop is enabled. The flip flop output is 1 with D= 1 and output is 0 with D = 0.
- Therefore, D Flip-Flop is said as Delay Flip-Flop or Data Flip-Flop or Transparent Flip-Flop.



Truth Table of D Flip-Flop

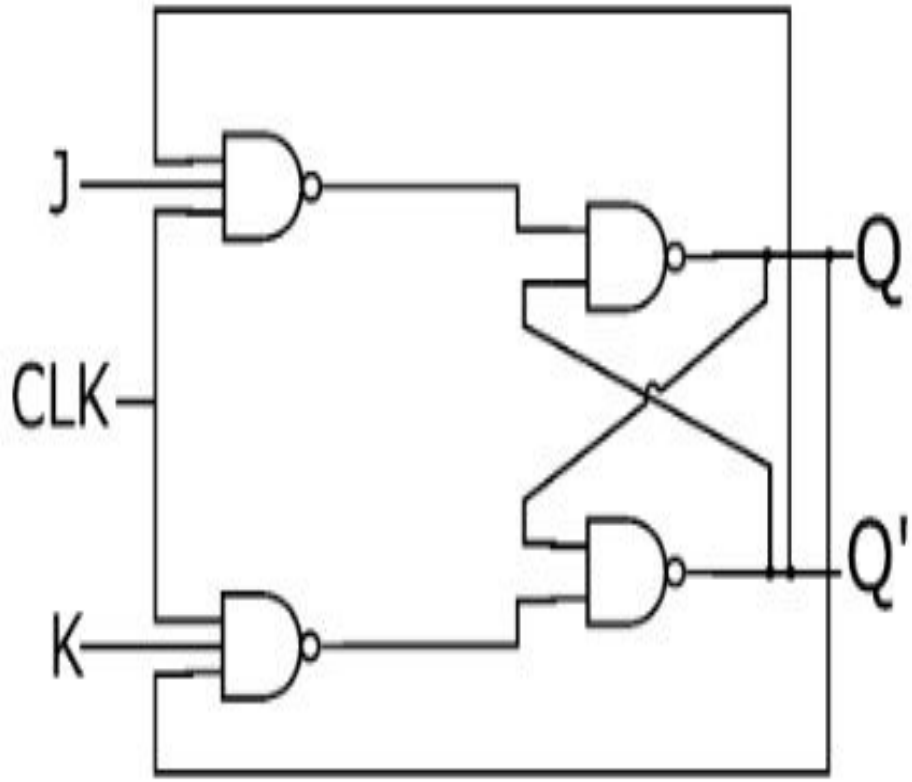
D	Q
0	0
1	1

Characteristics Equation of D Flip-Flops

$$Q(t+1) = D$$

J-K Flip-Flop

- The JK flip-flop operates with only positive or negative clock transitions.
- The operation of the JK flip-flop is similar to the SR flip-flop.
- When the inputs J and K are different, then the output Q takes the value of J at the next clock edge.
- When J and K both are low, then NO change occurs at the output.
- If both J and K are high, then at the clock edge, the output will toggle from one state to the other.


J
CLK
K
Q
Q'

Truth Table of JK Flip-Flop

J	K	Q	State
0	0	0	No Change
0	1	0	Reset
1	0	1	Set
1	1	Toggles	Toggle

Characteristics Table of JK Flip-Flop

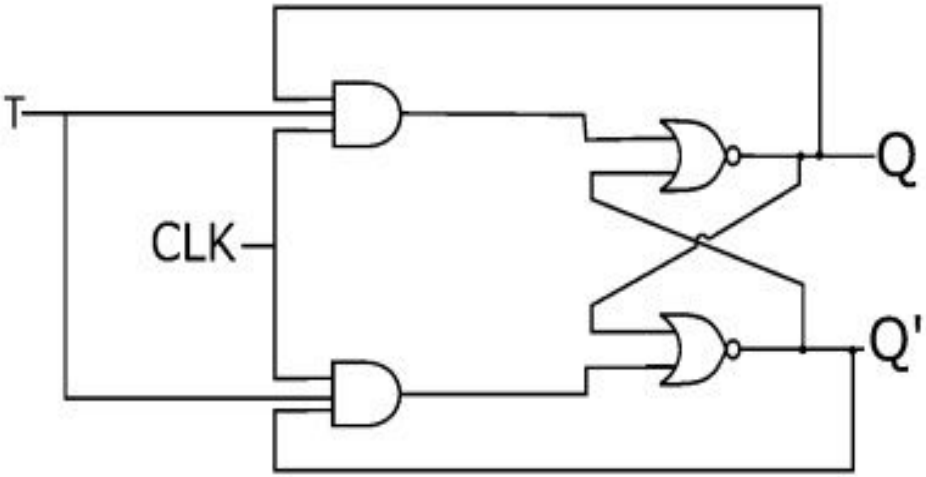
J	K	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Characteristics Equation of JK Flip-Flop

$$Q(t+1) = jkQ(t)' + K'Q(t)$$

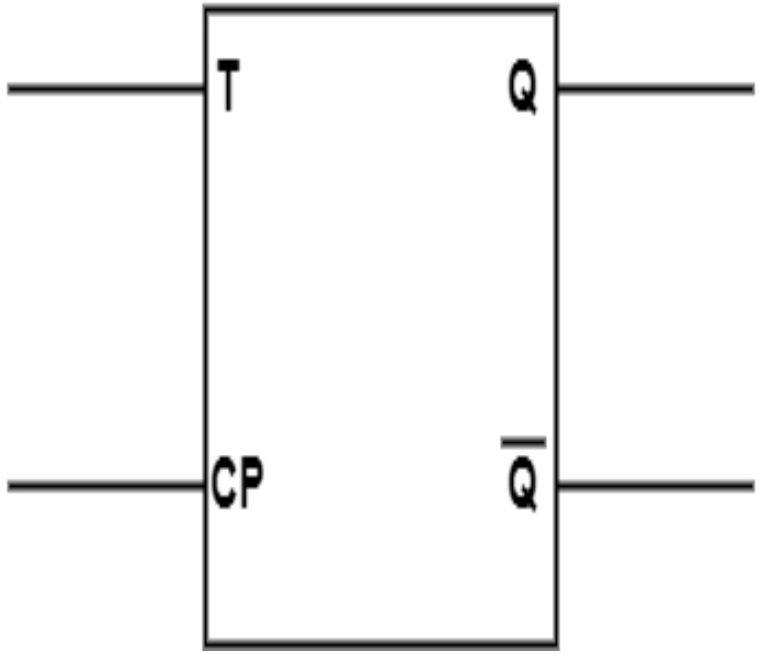
T Flip-Flop

- A T flip-flop (Toggle Flip-flop) is a simplified version of the JK flip-flop. The T flop is obtained by connecting the J and K inputs. The flip-flop has one input terminal and a clock input. These flip-flops are said to be T flip-flops because of their ability to toggle the input state. Toggle flip-flops are mostly used in counters.



Truth Table:

T	Q	Q(t+1)
0	0	0
1	0	1
0	1	1
1	1	0



Flip Flop Conversion

- Flip-flop conversion refers to the process of modifying one type of flip-flop (e.g., SR, D, JK, or T) to behave like another type by designing an appropriate input logic circuit.
- This is often required when a specific flip-flop is not directly available or for simplifying circuit design.

- **General Steps for Flip-Flop Conversion**
- **Understand the Source and Target Flip-Flops:**
  - Identify the type of flip-flop you have (source flip-flop) and the type you want to convert it into (target flip-flop).
  - Know the characteristic equations and truth tables for both types.
- **Write the Truth Table for the Target Flip-Flop:**
  - Create a truth table for the target flip-flop based on its characteristic equation and behaviour. Include the current state (Q), the input of the target flip-flop, and the next state (Qnext).
- **Relate the Target and Source Flip-Flops:**
  - For each possible combination of Q (current state) and Qnext (desired next state), determine the required inputs for the source flip-flop to achieve the same behaviour.

- **Determine the Input Expressions:**
  - Derive the logic expressions for the inputs of the source flip-flop that will allow it to produce the same output transitions as the target flip-flop.
  - Use Karnaugh maps (K-maps) or Boolean algebra to simplify the input logic expressions.
- **Design the Conversion Logic Circuit:**
  - Use the simplified expressions to design a circuit using logic gates that connect the input(s) of the source flip-flop to behave like the target flip-flop.
- **Verify the Conversion:**
  - Validate the conversion by ensuring that the source flip-flop with the designed logic produces the same Qnext as the target flip-flop for all input conditions.

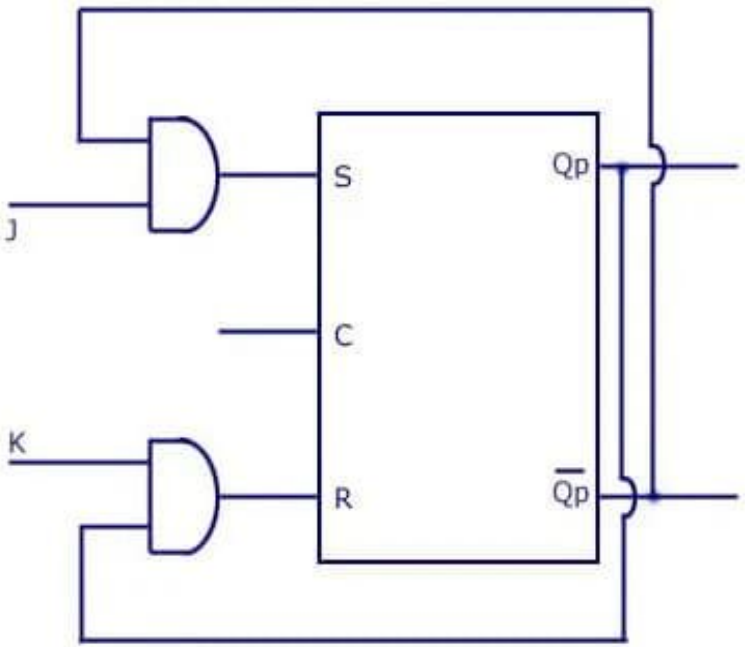
Converting SR Flip-Flop to JK Flip-Flop

- Start by creating a truth table for J, K, and the present state (QP), considering all possible combinations.
- For each combination, determine the next state (QN).
- Identify the S and R values needed to transition from QP to QN.
- Using the truth table, the Boolean equations for S and R are derived using Karnaugh maps

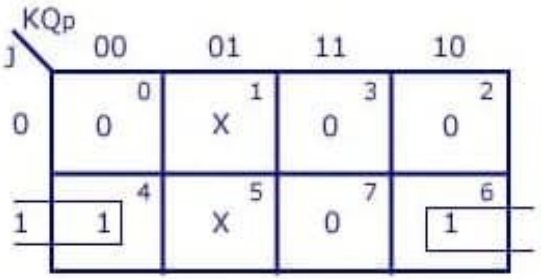
Conversion Table

J-K Inputs		Outputs		S-R Inputs	
J	K	Qp	Qp+1	S	R
0	0	0	0	0	X
0	0	1	1	X	0
0	1	0	0	0	X
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	X	0
1	1	0	1	1	0
1	1	1	0	0	1

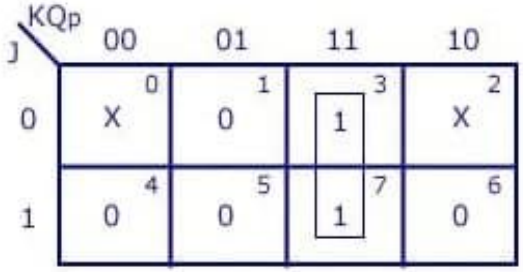
Logic Diagram



K-Map



$S = \overline{J}Qp$



$R = KQp$

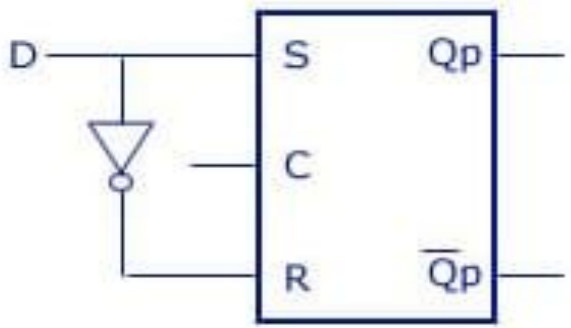
S-R Flip Flop to J-K Flip Flop

- **Converting SR Flip-Flop to D Flip-Flop**
- Transforming an SR flip-flop into a D flip-flop is simpler.
- The Data (D) input is connected to the S input, while its inverted form (D’) is connected to the R input.
- The Boolean equations derived from the truth table are:
- S=D
- R=D’

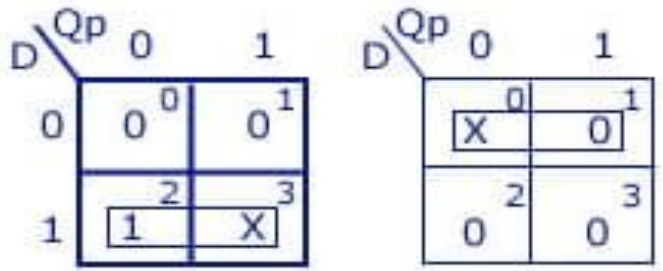
Conversion Table

D Input	Outputs		S-R Inputs	
	Qp	Qp+1	S	R
0	0	0	0	X
0	1	0	0	1
1	0	1	1	0
1	1	1	X	0

Logic Diagram



K-maps



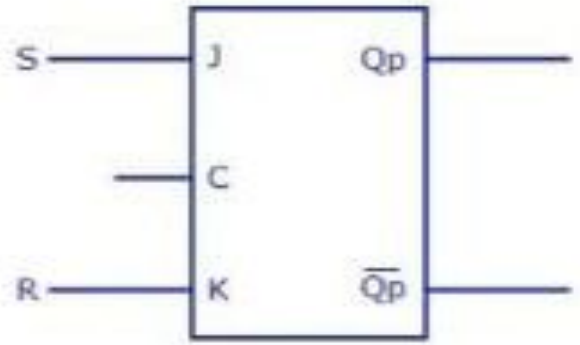
S-R Flip Flop to D Flip Flop

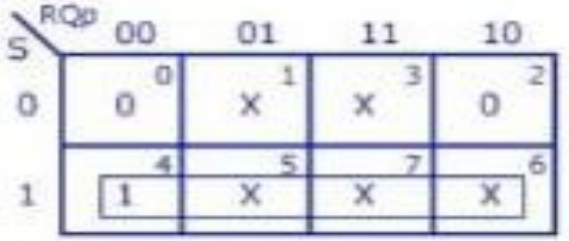
- **Converting JK Flip-Flop to SR Flip-Flop**
- The JK flip-flop can be directly converted into an SR flip-flop because the logic for J and K aligns with S and R.
- J=S
- K=R
- No additional circuitry is needed—simply relabel the inputs.

Conversion Table

S-R Inputs		Outputs		J-K Inputs	
S	R	Qp	Qp+1	J	K
0	0	0	0	0	X
0	0	1	1	X	0
0	1	0	0	0	X
0	1	1	0	X	1
1	0	0	1	1	X
1	0	1	1	X	0
1	1	Invalid		Dont care	
1	1	Invalid		Dont care	

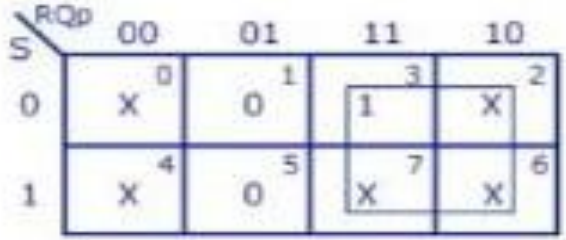
Logic Diagram





J=S

K-maps



K=R

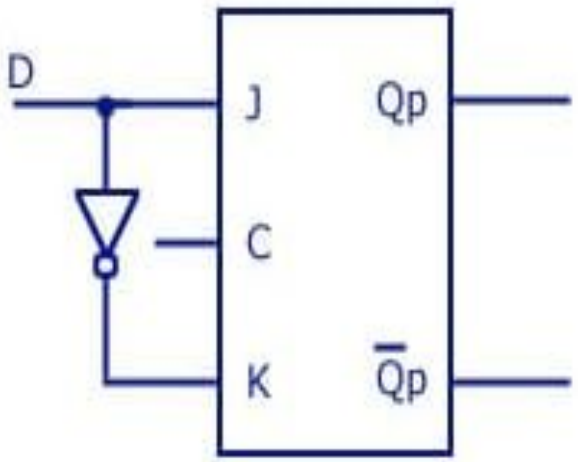
J-K Flip Flop to S-R Flip Flop

- **Converting JK Flip-Flop to D Flip-Flop**
- To transform a JK flip-flop into a D flip-flop:
- The Data (D) input connects directly to J.
- The inverted D (D’) connects to K.
- The Boolean equations are straightforward:
- J=D
- K=D’

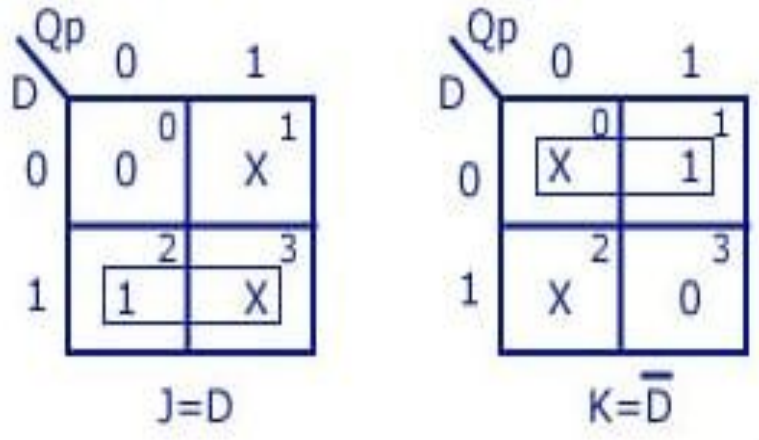
Conversion Table

D Input	Outputs		J-K Inputs	
	Qp	Qp+1	J	K
0	0	0	0	X
0	1	0	X	1
1	0	1	1	X
1	1	0	X	0

Logic Diagram



K-maps



J-K Flip Flop to D Flip Flop

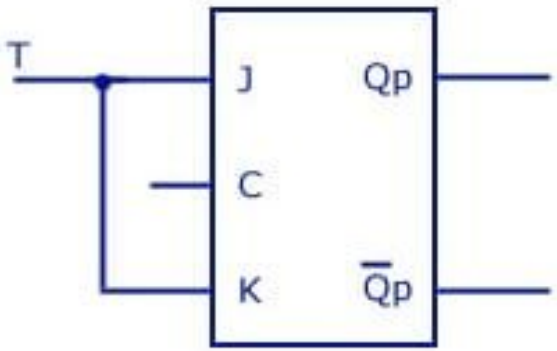
- **Converting JK Flip-Flop to T Flip-Flop**
- Converting a JK flip-flop into a T flip-flop is as simple as connecting the T input directly to both J and K.
- J=T; K=T

Conversion Table

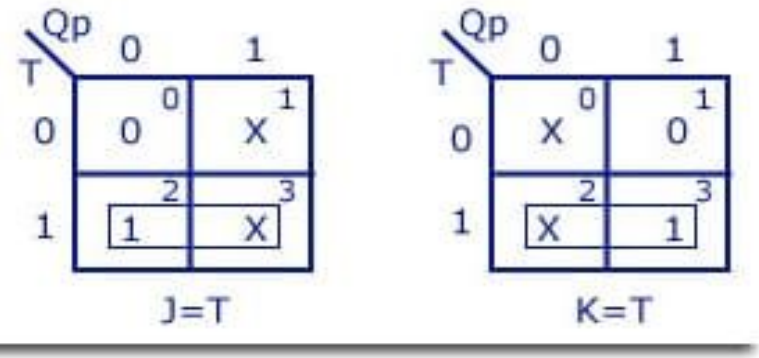
T Input	Outputs		J-K Inputs	
	Qp	Qp+1	J	K
0	0	0	0	X
0	1	1	X	0
1	0	1	1	X
1	1	0	X	1

J-K Flip Flop to T Flip Flop

Logic Diagram



K-maps



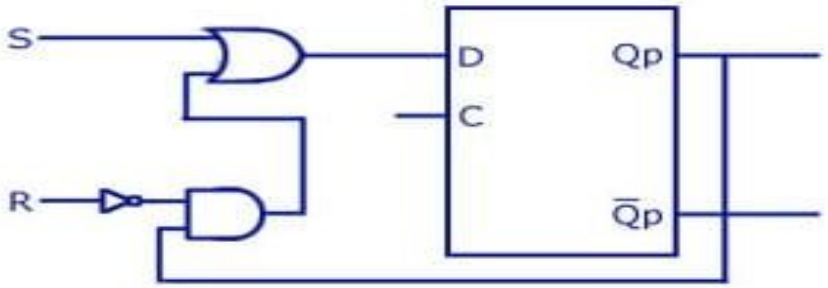
- **Converting D Flip-Flop to SR Flip-Flop**
- To convert a D flip-flop to an SR flip-flop, a combinational circuit is added to derive S and R from D.
- Truth tables show:
  - S=D; R=D'

Conversion Table

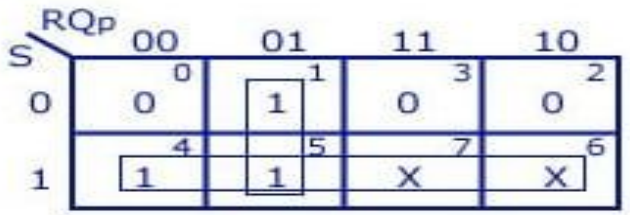
S-R Inputs S	R	Outputs Qp	Qp+1	D Input
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	Invalid		Dont care
1	1	Invalid		Dont care

D Flip Flop to S-R Flip Flop

Logic Diagram



K-map



$D = S + \overline{R}Q_n$

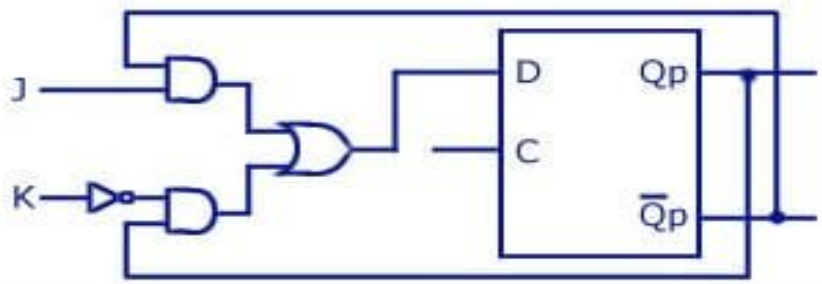
- **Converting D Flip-Flop to JK Flip-Flop**
- The conversion involves creating J and K inputs based on the D input.
- J=D; K=D’

Conversion Table

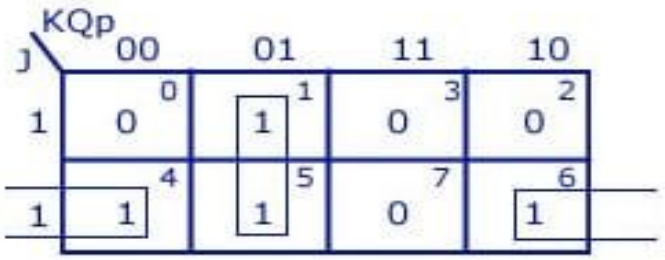
J-K Input J	J-K Input K	Outputs Qp	Outputs Qp+1	D Input
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

D Flip Flop to J-K Flip Flop

Logic Diagram



K-map



$D = J\overline{Q}p + \overline{K}Qp$

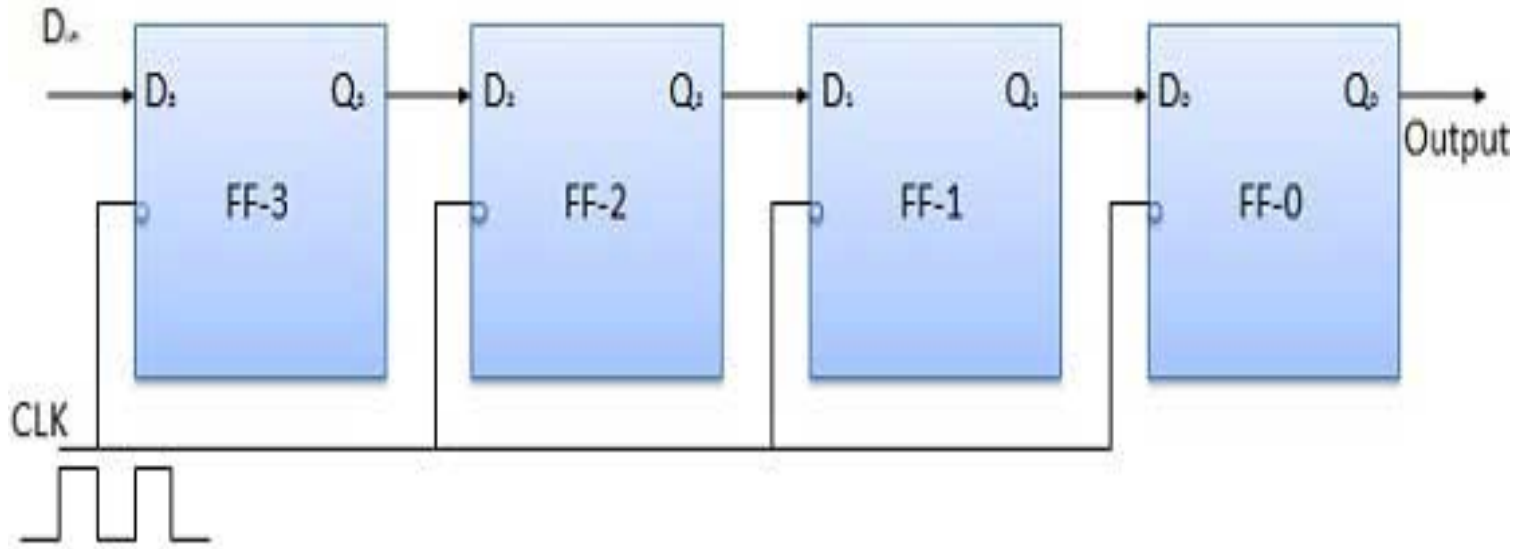
shift registers

- A flip-flop is a 1-bit memory cell that can be used for storing digital data. To increase the storage capacity in terms of the number of bits, a group of flip-flops is used.
- Such a group of flip-flops is known as a **Register**.
- The **n-bit register** will consist of **n** flip-flops, and it is capable of storing an **n-bit** word.
- The binary data in a register can be moved within the register from one flip-flop to another.
- The registers that allow such data transfers are called **shift registers**. There are four modes of operation of a shift register.

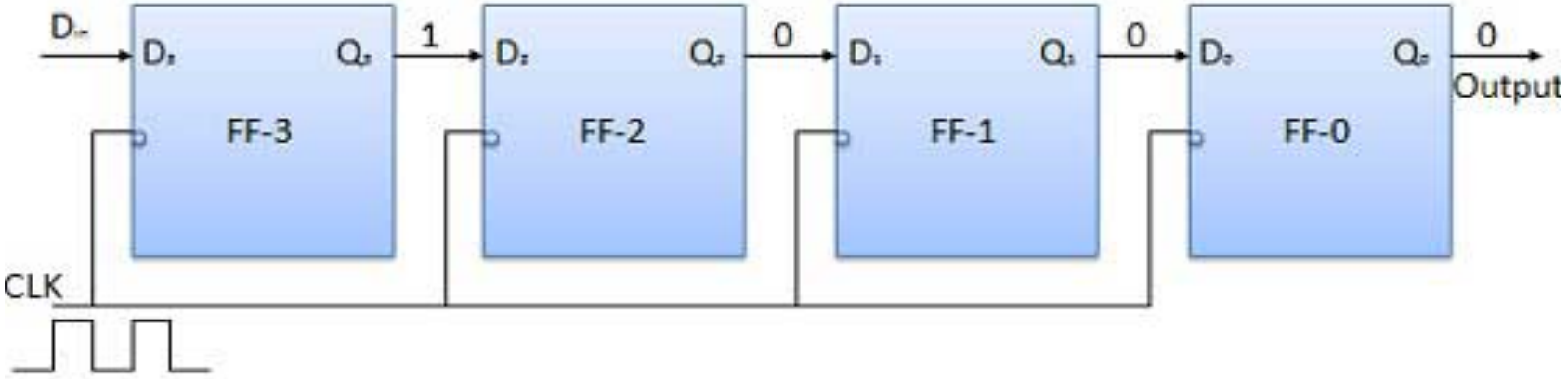
- Serial In - Serial Out (SISO) Shift Register
- Serial In - Parallel Out (SIPO) Shift Register
- Parallel In - Serial Out (PISO) Shift Register
- Parallel In - Parallel Out (PIPO) Shift Register

Serial In - Serial Out (SISO) Shift Register

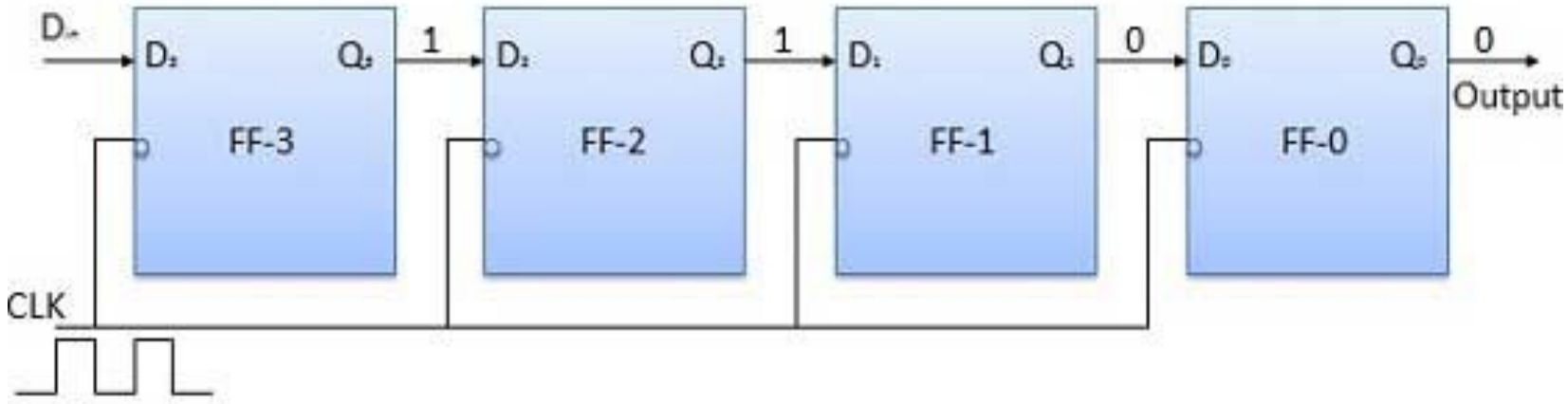
- Let all the flip-flops be initially in the reset condition, i.e., $Q_3 = Q_2 = Q_1 = Q_0 = 0$.
- If an entry of a four-bit binary number 1 1 1 1 is made into the register, this number should be applied to $\mathbf{D_{in}}$ bit with the LSB bit applied first.
- The D input of FF-3, i.e., D_3 is connected to serial data input $\mathbf{D_{in}}$.
- Output of FF-3, i.e., Q_3 is connected to the input of the next flip-flop, i.e., D_2 , and so on.

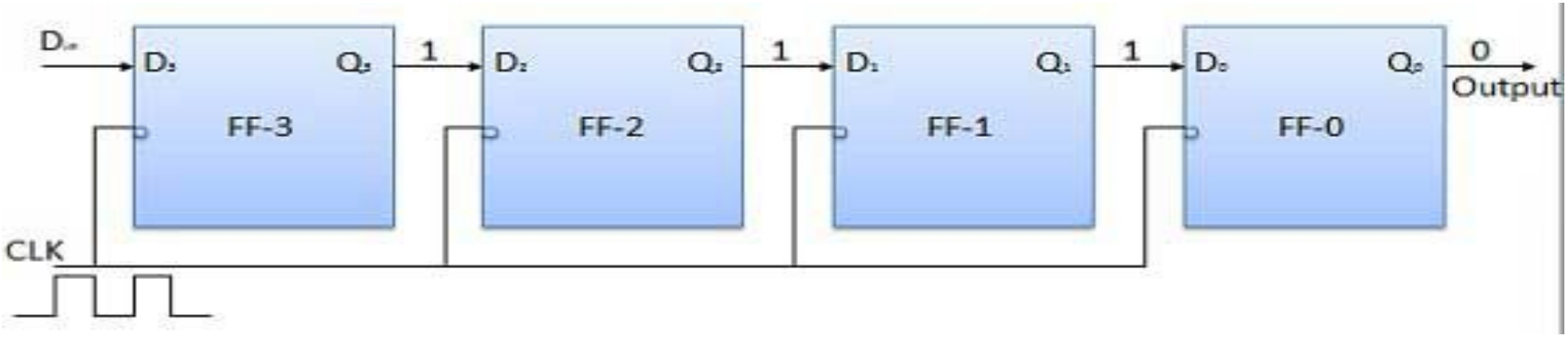


- Before application of the clock signal, let $Q_3\ Q_2\ Q_1\ Q_0$ = 0000 and apply the LSB bit of the number to be entered to D_{in}. So $D_{in} = D_3 = 1$.
- Apply the clock. On the first falling edge of the clock, the FF-3 is set, and the stored word in the register is $Q_3\ Q_2\ Q_1\ Q_0$ = 1000.

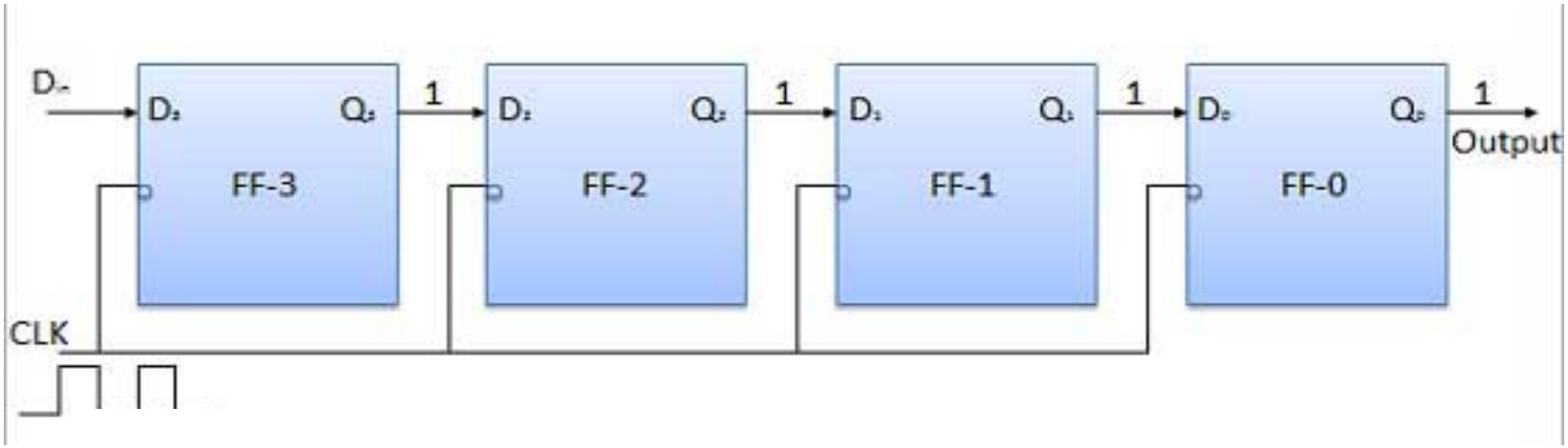


- Apply the next bit to D_{in}. So D_{in} = 1. As soon as the next negative edge of the clock hits, FF-2 will set, and the stored word changes to $Q_3\,Q_2\,Q_1\,Q_0$ = 1100.



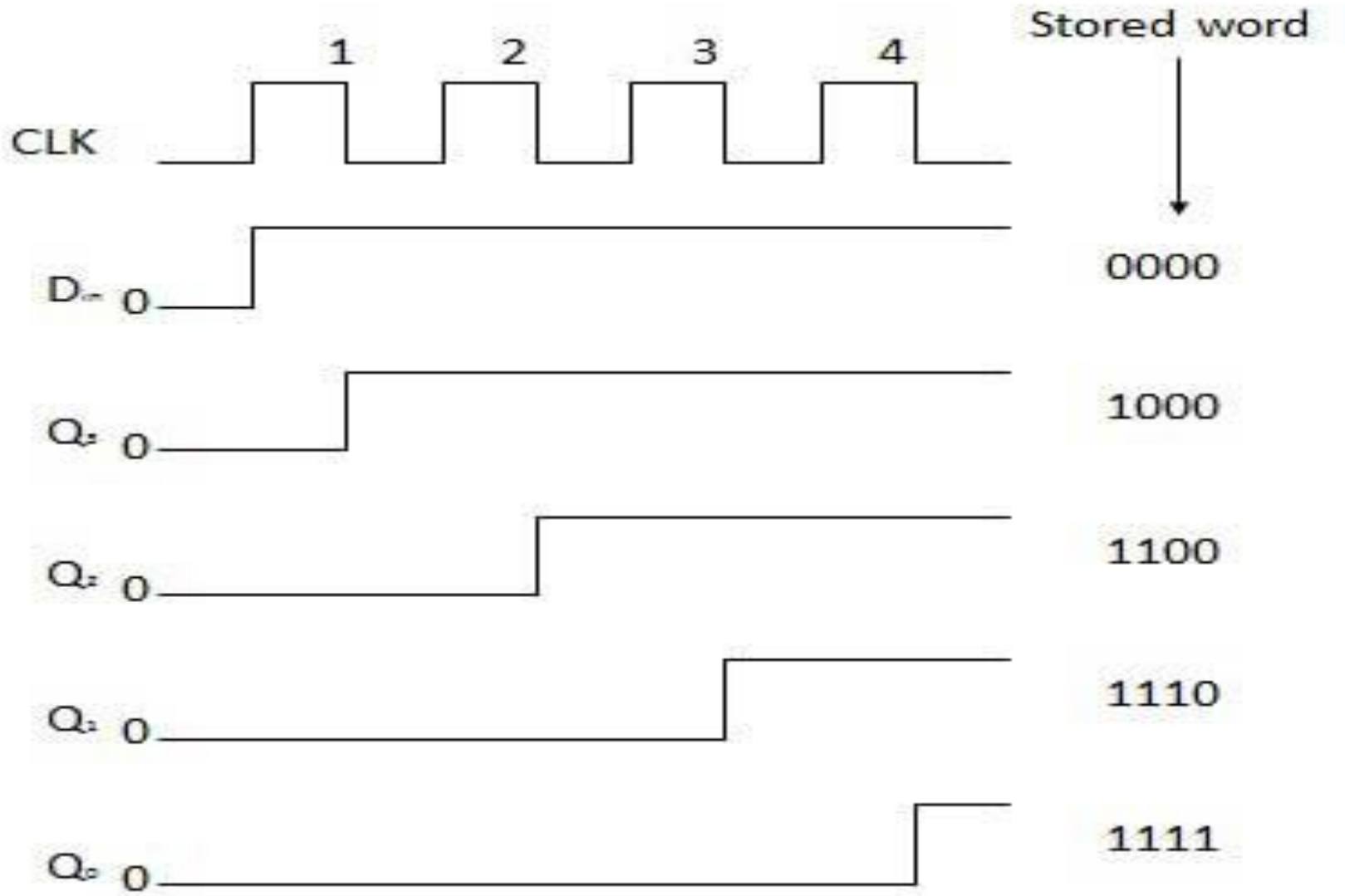


- Apply the next bit to be stored, i.e., 1 to D_{in}. Apply the clock pulse. As soon as the third negative clock edge hits, FF-1 will be set, and output will be modified to $Q_3\ Q_2\ Q_1\ Q_0 = 1110$.
- Similarly, with $D_{in} = 1$ and with the fourth negative clock edge arriving, the stored word in the register is $Q_3\ Q_2\ Q_1\ Q_0 = 1111$.



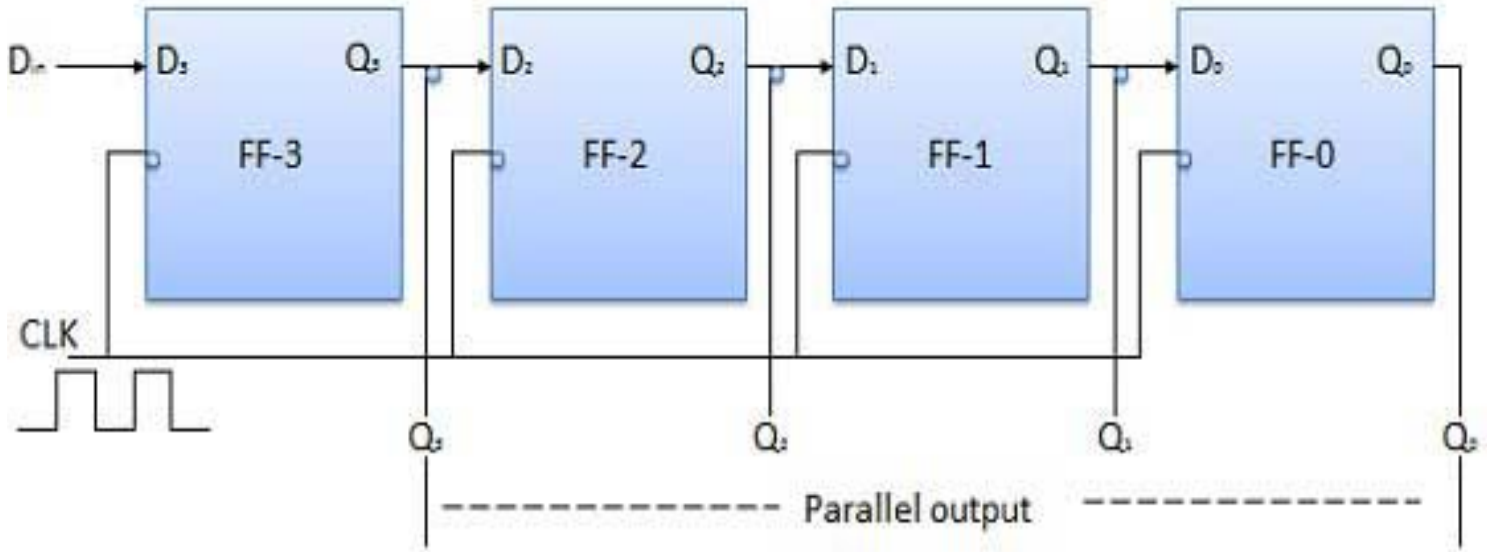
	CLK	$D_n = Q_3$	$Q_3 = D_2$	$Q_2 = D_1$	$Q_1 = D_0$	Q_0
Initially			0	0	0	0
(i)	↓	1	1	0	0	0
(ii)	↓	1	1	1	0	0
(iii)	↓	1	1	1	1	0
(iv)	↓	1	1	1	1	1

→ Direction of data travel


Stored word
1
2
3
4
CLK
0
0
0
0
0
0000
1000
1100
1110
1111

Serial In - Parallel Out (SIPO) Shift Register

- In such types of operations, the data is entered serially and taken out in a parallel fashion.
- Data is loaded bit by bit. The outputs are disabled as long as the data is loading.
- As soon as the data loading gets completed, all the flip-flops contain their required data, and the outputs are enabled so that all the loaded data is made available over all the output lines at the same time.
- 4 clock cycles are required to load a four-bit word. Hence, the speed of operation of SIPO mode is the same as that of SISO mode.


D
D3
Q3
FF-3
D2
Q2
FF-2
D1
Q1
FF-1
D0
Q0
FF-0
CLK
Q3
Q2
Q1
Q0
Parallel output

Parallel In - Serial Out (PISO) Shift Register

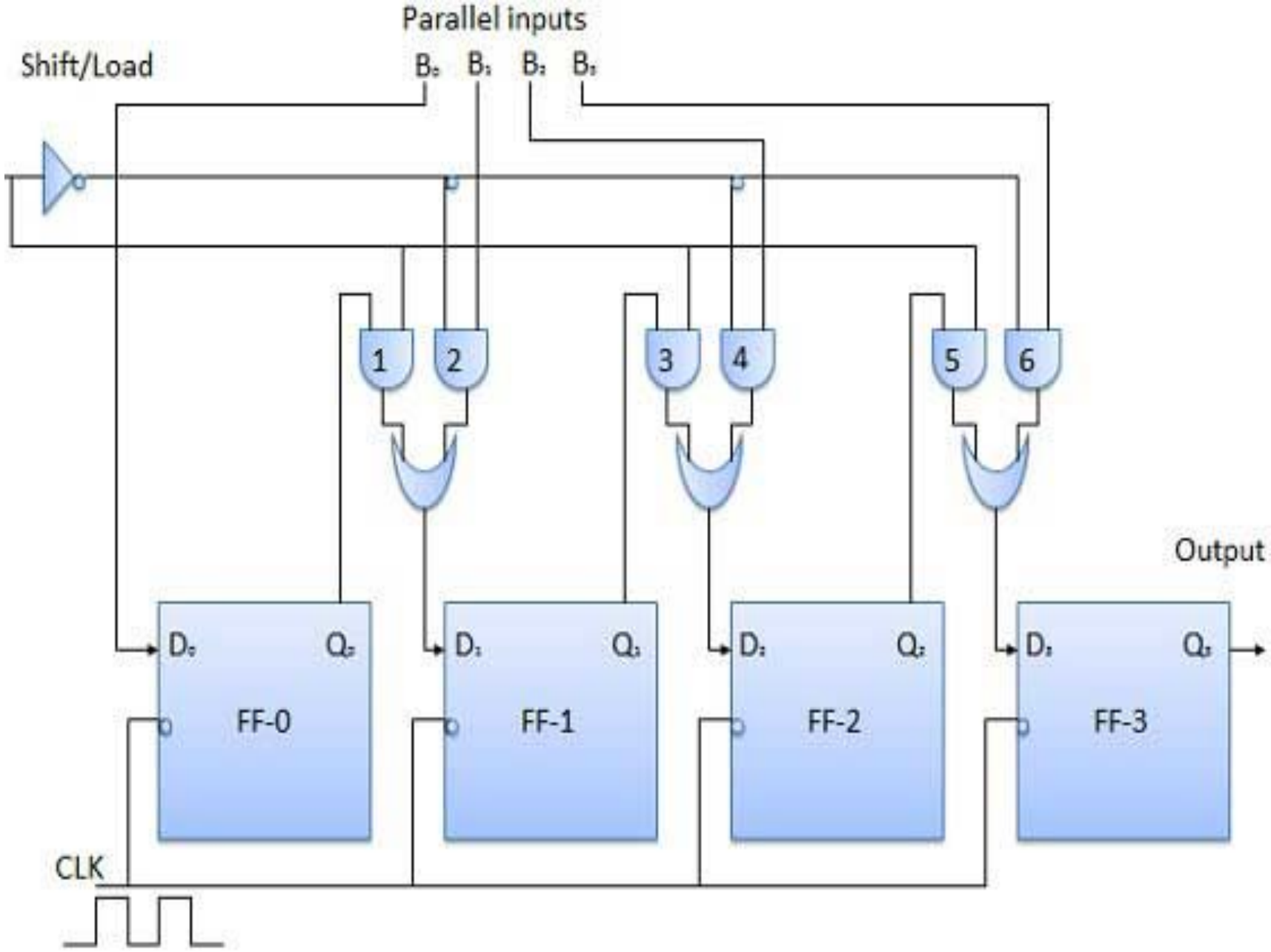
- Data bits are entered in a parallel fashion.
- The circuit shown below is a four-bit parallel input serial output register.
- Output of the previous flip-flop is connected to the input of the next one via a combinational circuit.
- The binary input word B_0, B_1, B_2, B_3 is applied though the same combinational circuit.
- There are two modes in which this circuit can work, namely - shift mode or load mode.

Load Mode

- When the shift/load bar line is low (0), the AND gates 2, 4, and 6 become active they will pass B_1, B_2, B_3 bits to the corresponding flip-flops.
- On the low-going edge of the clock, the binary inputs B_0, B_1, B_2, B_3 will get loaded into the corresponding flip-flops. Thus, parallel loading takes place.

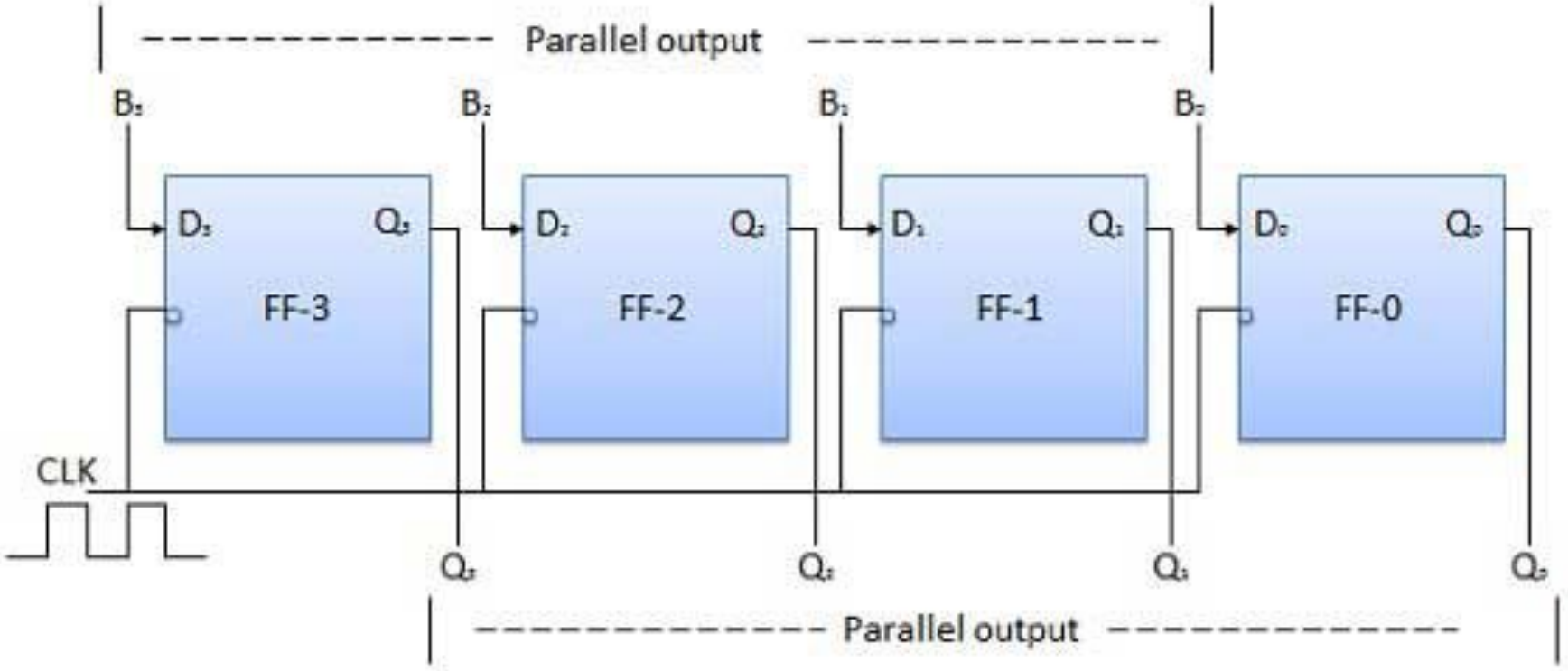
Shift Mode

- When the shift/load bar line is low (1), the AND gates 2, 4, and 6 become inactive. Hence, the parallel loading of the data becomes impossible.
- But the AND gates 1,3, and 5 become active. Therefore, the shifting of data from left to right bit by bit on the application of clock pulses. Thus, the parallel in serial out operation takes place.


Parallel inputs
Shift/Load
B0 B1 B2 B3
1
2
3
4
5
6
Output
D0 Q0
FF-0
D1 Q1
FF-1
D2 Q2
FF-2
D3 Q3
FF-3
CLK

Parallel In - Parallel Out (PIPO) Shift Register

- In this mode, the 4 bit binary input B_0, B_1, B_2, B_3 is applied to the data inputs D_0, D_1, D_2, D_3 respectively of the four flip-flops.
- As soon as a negative clock edge is applied, the input binary bits will be loaded into the flip-flops simultaneously.
- The loaded bits will appear simultaneously on the output side. Only a clock pulse is essential to load all the bits.


Parallel output
B3
B2
B1
B0
D3
Q3
FF-3
D2
Q2
FF-2
D1
Q1
FF-1
D0
Q0
FF-0
CLK
Q3
Q2
Q1
Q0
Parallel output

Counters

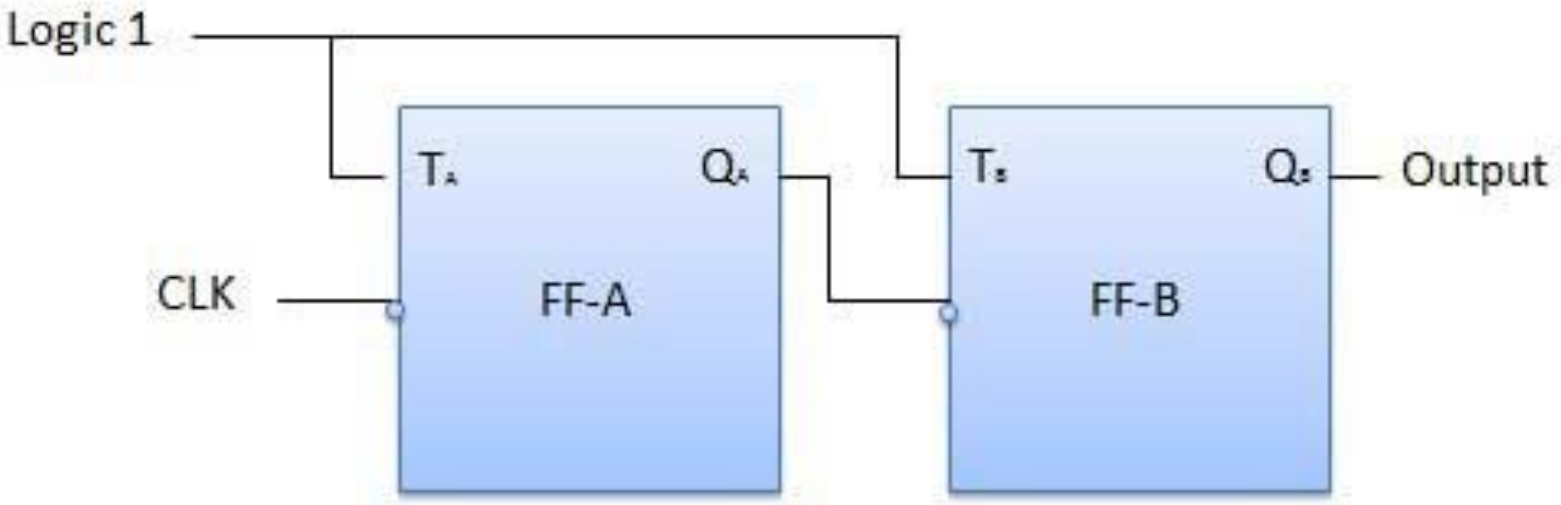
- A digital circuit that is used for counting pulses is known counter. The counter is the widest application of flip-flops. It is a group of flip-flops with a clock signal applied.

Counters are of two types:

- Asynchronous or Ripple Counters
- Synchronous Counters

Asynchronous or Ripple Counters

- The toggle (T) flip-flop is being used. The JK flip-flop can also be used with J and K permanently connected to logic 1.
- An external clock is applied to the flip-flop A's clock input, and the QA output is applied to the next flip-flop's clock input, i.e., FF-B.



Operation

- Initially, let both the FFs be in the reset state, then $Q_B Q_A$ = 00 initially.

After 1st negative clock edge

- As soon as the first negative clock edge is applied, FF-A will toggle, and Q_A will be equal to 1.
- Q_A is connected to the clock input of FF-B. Since Q_A has changed from 0 to 1, it is treated as the positive clock edge by FF-B. There is no change in Q_B because FF-B is a negative edge-triggered FF.
- $Q_B Q_A$ = 01 after the first clock pulse.

After the 2nd negative clock edge

- On the arrival of the second negative clock edge, FF-A toggles again and $Q_A = 0$.
- The change in Q_A acts as a negative clock edge for FF-B. So it will also toggle, and Q_B will be 1.
- $Q_B Q_A = 10$ after the second clock pulse.

After the 3rd negative clock edge

- On the arrival of the 3rd negative clock edge, FF-A toggles again, and Q_A becomes 1 from 0.
- Since this is a positive change, FF-B does not respond to it and remains inactive. So Q_B does not change and continues to be equal to 1.
- $Q_B Q_A = 11$ after the third clock pulse.

After the 4th negative clock edge

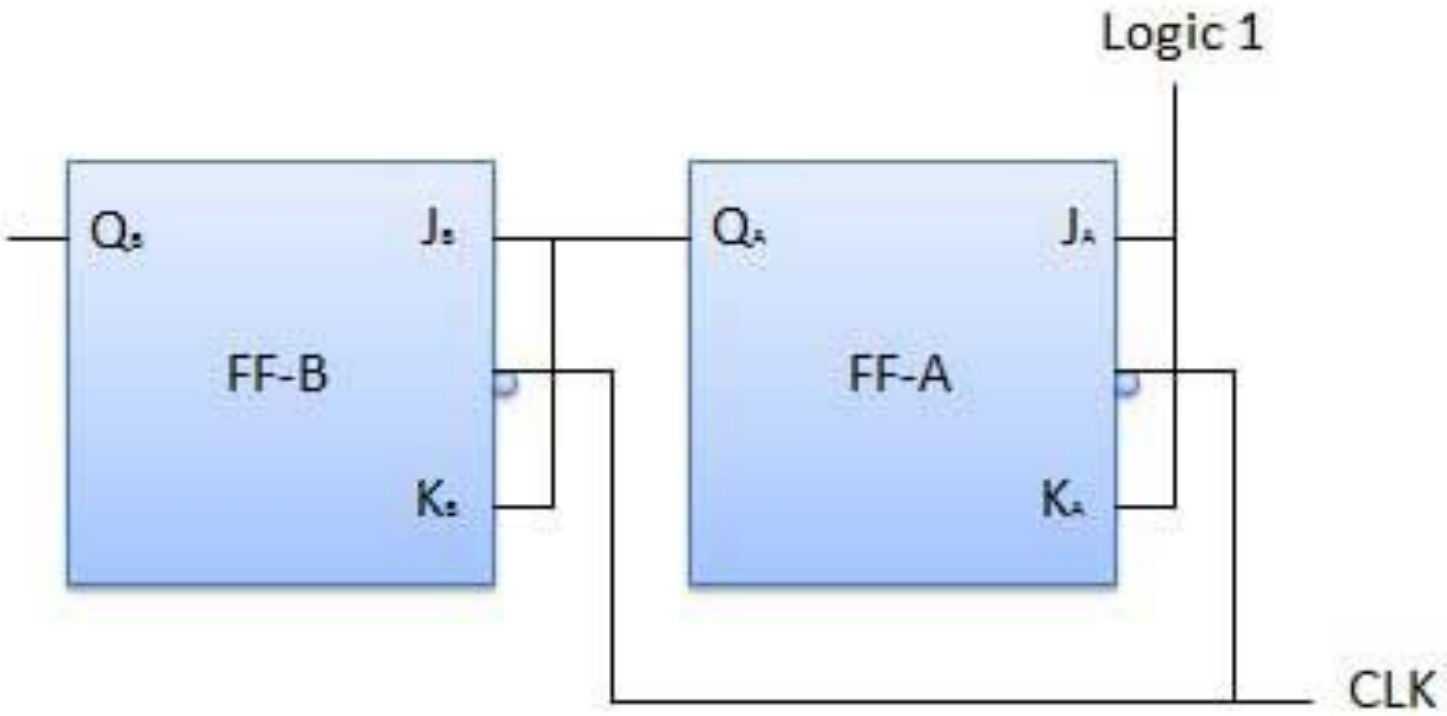
- On the arrival of the 4th negative clock edge, FF-A toggles again, and Q_A becomes 1 from 0.
- This negative change in Q_A acts as a clock pulse for FF-B. Hence, it toggles to change Q_B from 1 to 0.
- Q_BQ_A = 00 after the fourth clock pulse.

Truth Table

Clock	Counter output		State number	Deciimal Counter output
	Q_B	Q_A		
Initially	0	0	—	0
1st	0	1	1	1
2nd	1	0	2	2
3rd	1	1	3	3
4th	0	0	4	0

Synchronous Counters

- If the "clock" pulses are applied to all the flip-flops in a counter simultaneously, then such a counter is called a synchronous counter.



Initially, let both the FFs be in the reset state

- $Q_B Q_A$ = 00 initially.

After 1st negative clock edge

- As soon as the first negative clock edge is applied, FF-A will toggle, and Q_A will change from 0 to 1.
- But at the instant of application of the negative clock edge, Q_A, $J_B = K_B = 0$. Hence, FF-B will not change its state. So Q_B will remain 0.
- $Q_B Q_A$ = 01 after the first clock pulse.

After the 2nd negative clock edge

- On the arrival of the second negative clock edge, FF-A toggles again and Q_A changes from 1 to 0.
- But at this instant, Q_A was 1. So $J_B = K_B = 1$, and FF-B will toggle. Hence, Q_B changes from 0 to 1.
- $Q_B Q_A$ = 10 after the second clock pulse.

After 3rd negative clock edge

- On application of the third falling clock edge, FF-A will toggle from 0 to 1, but there is no change of state for FF-B.
- $Q_B Q_A$ = 11 after the third clock pulse.

After the 4th negative clock edge

- On application of the next clock pulse, Q_A will change from 1 to 0 as Q_B will also change from 1 to 0.
- $Q_B Q_A$ = 00 after the fourth clock pulse.

Types of Binary Counters

- **Asynchronous Counter** − The type of binary counter in which the flip-flops do not receive the same clock pulse at the same time is called an asynchronous counter.
- The asynchronous counter is also known as a **ripple counter**. It is the simplest type of binary counter.
- In the case of an asynchronous binary counter, each flip-flop is triggered by the output of the previous flip-flop.
- Therefore, the asynchronous counters suffer from propagation delay.

- **Synchronous Counter** – The type of binary counter in which all the flip-flops receive the same clock pulse at the same time is known as a synchronous counter.
- Since the same clock pulse triggers all the flip-flops of the synchronous counter, their outputs change simultaneously.
- This will result in no propagation delay between the flip-flops.
- **Up Counter** – The type of binary counter that counts upwards from zero to its maximum count value is known as an up counter.
- In the case of an up counter, the count is increased by one on each clock pulse.

- **Down Counter** – The type of binary counter that counts downwards from its maximum count value to zero is known as a down counter. In the down counter, the count value of the counter is decreased by one on each clock pulse.
- **Up/Down Counter** – The type of binary counter that can count in both upward and downward directions is known as an up/down counter. In the up/down counter, the direction of count is determined by a control input signal.

4-bit binary Adder-Subtractor

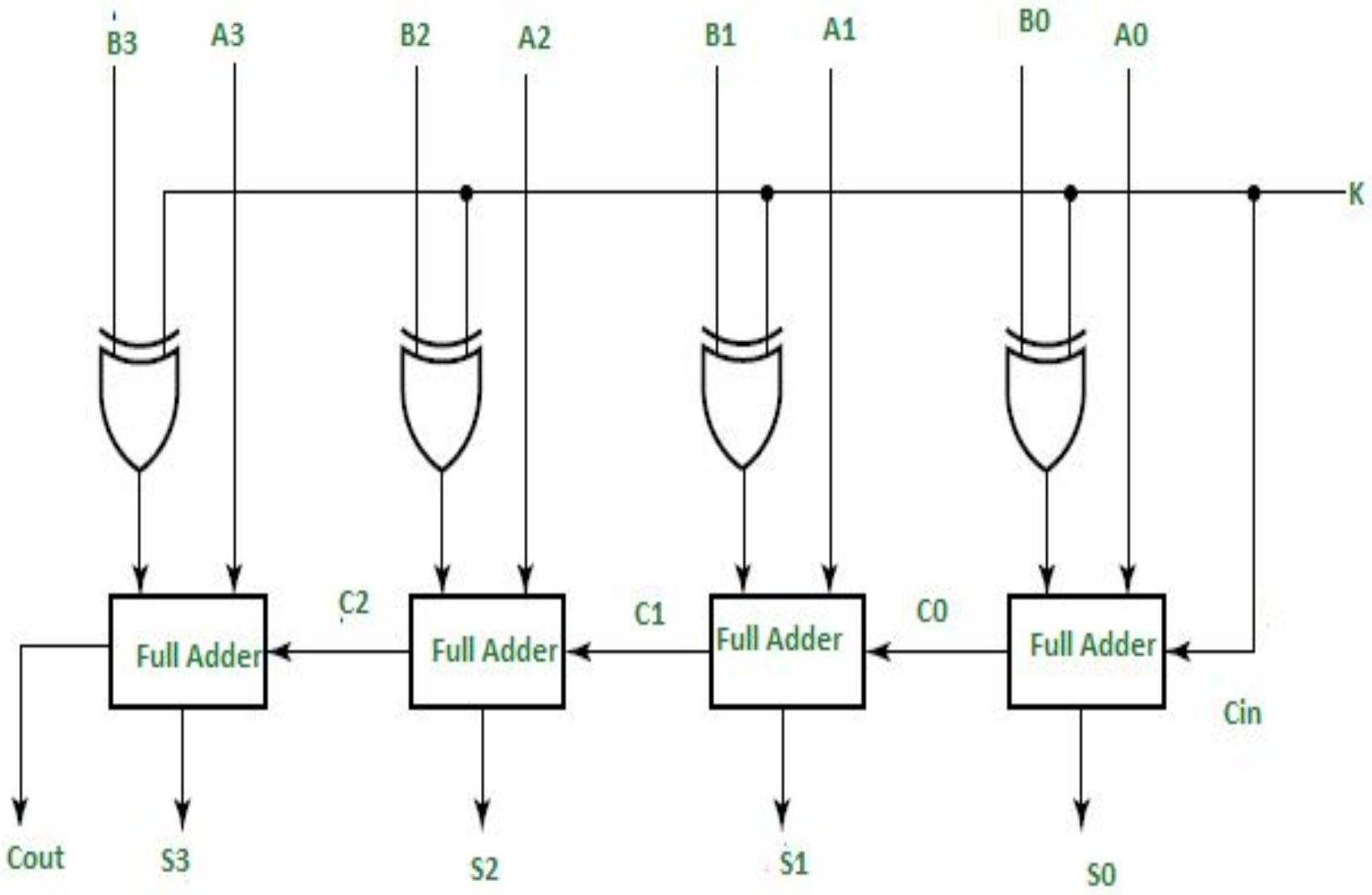
- A **Binary Adder-Subtractor** is can do both the addition and subtraction of binary numbers in one circuit itself. The operation is performed depending on the binary value the control signal holds. It is one of the components of the ALU (Arithmetic Logic Unit).

What is a Binary Adder?

- A **Binary Adder** is an electronic circuit that sums two binary numbers. The most basic types are half adders and full adders. A half adder takes in two one-bit numbers, producing a sum and a carry bit.
- Full adder has three input bits: two actual bits and an incoming carry from the preceding operation. This makes the Full Adder to add multi-bit binary numbers as the carry can propagate through several Full Adders in a ripple-carry adder configuration.

What is Binary Subtractor?

- A **Binary Subtractor** is used to perform the subtraction of two binary numbers. Just as with addition, this can also be done using an array of logic gates.
- The subtraction is done by using two's complement, in which the circuit first negates the binary number to obtain its complement and then adds one to the result before inputting it into a binary adder circuit together with the original number.
- Both addition and subtraction are handled by this type of circuit.
- The circuit consists of 4 full adders since we are performing operations on 4-bit numbers.
- There is a control line K that holds a binary value of either 0 or 1, which determines whether the operation is carried out is addition or subtraction.


B3
A3
B2
A2
B1
A1
B0
A0
K
Full Adder
Full Adder
Full Adder
Full Adder
C2
C1
C0
Cin
Cout
S3
S2
S1
S0

- Consider two 4-bit binary numbers A and B as inputs to the Digital Circuit for the operation with digits
- A0 A1 A2 A3 for A
- B0 B1 B2 B3 for B
- The circuit consists of 4 full adders since we are performing operations on 4-bit numbers. There is a control line K that holds a binary value of either 0 or 1, which determines whether the operation is carried out as addition or subtraction.
- The first full adder has a control line directly as its input(input carry Cin). The input A0 (The least significant bit of A) is directly input into the full adder. The third input is the **xor** of B0 and K. The two outputs produced are Sum/Difference (S0) and Carry (C0).

- When K=1, the operation being performed on the four-bit numbers is subtraction.
- If the value of K (Control line) is 1, the output of B0(exor)K=B0'(Complement B0). Thus, the operation would be A+(B0'). Now, 2's complement subtraction for two numbers A and B is given by A+B'+Cin.
- If the Value of K=0, B0 (exor) K=B0. The operation is A+B, which is simple binary addition. This suggests that when K=0, the operation is performed on the four-bit numbers in addition.

- Then C0 is serially passed to the second full adder as one of its outputs. The sum/difference S0 is recorded as the least significant bit of the sum/difference.
- A1, A2, and A3 are direct inputs to the second, third, and fourth full adders.
- Then the third input is the B1, B2, B3 XORed with K to the second, third and fourth full adder, respectively.
- The carry C1, C2 are serially passed to the successive full adder as one of the inputs.
- C3 becomes the total carry to the sum/difference. S1, S2, S3 are recorded to form the result with S0.

Example: Assume that we have two 3-bit numbers, i.e., X=100 and Y=011, and feed them in Full-Adder as an input.

- X0 = 0 X1 = 0 X2 = 1
- Y0 = 1 Y1 = 1 & Y2 = 0
- **For K=0:**
- Y0⊕K=Y0 and Cin=K=0
- So, from the first Full-Adder
- S0 = X0+Y0+Cin
- S0= 0+1+0
- S0=1
- C0=0

- Similarly,
- S1 = X1+Y1+C0
- S1 = 0+1+0
- S1=1 and C1=0
- Similarly,
- S2 = X2+Y2+C1
- S2 = 1+0+0
- S2=1 and C2=0

- Thus,

- X= 100 =4
- Y = 011 = 3
- Sum = 0111 = 7

- For K=1
- Y0⊕K=Y0' and Cin=k=1

So,

- S0 = X0+Y0'+Cin
- S0 = 0+0+1
- S0=1 and C0=0

Similarly,

- S1 = X1+Y1'+C0
- S1 = 0+0+0
- S1=0 and C1=0

- Similarly,

- S2 = X2+Y2'+C1
- S2 = 1+1+0
- S2=0 and C2=0

- Thus,

- X = 010 = 4
- Y = 011 = 3

- Difference = 001 = 1

Semiconductor RAM memories

- A type of memory built using **semiconductor-based integrated circuits**.
- It is faster and more reliable compared to magnetic or mechanical memory.
- Mainly divided into:

Volatile memory → Loses data when power is off (e.g., RAM).

Non-volatile memory → Retains data without power (e.g., ROM).

- **RAM (Random Access Memory):**
- Temporary, high-speed memory that stores data and instructions while the CPU is working.
- **Characteristics:**
- Volatile (data is erased when power is off).
- Read and write operations are possible.
- Acts as main memory in computer systems.

Types of RAM

SRAM (Static RAM):

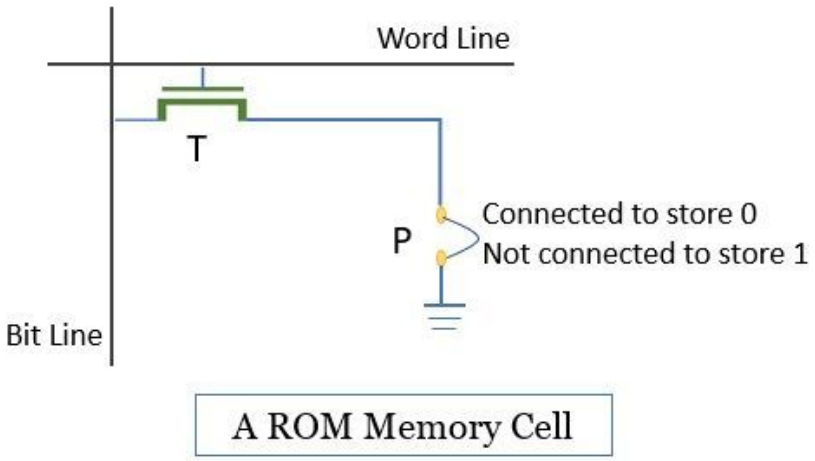
- Built using flip-flops.
- High-speed, low latency.
- More expensive and consumes more power.
- Used in Cache Memory.

DRAM (Dynamic RAM):

- Built using capacitors.
- Needs periodic refresh (dynamic).
- Slower, cheaper, high-density.
- Used in Main Memory.

Read-Only Memory (ROM)

- Read-only memory (ROM) is a non-volatile memory, which means the memory cells of this memory chip do not require a power supply to retain their bit value.
- As this is a read-only memory, the bit values of this memory can only be read and not be written or modified.



A ROM Memory Cell

- The figure below shows the structure of a ROM memory cell, where the bit value of the memory cell is 0 if the transistor is at a ground level, otherwise it is 1.
- The bit line is connected to the power supply via a resistor.
- To read the value of the memory cell, the word line is activated, which connects the transistor to the ground.

- This drops the voltage of the bit line to 0 if the transistor is connected to the ground.
- If there is no connection between the transistor and ground, the bit line remains at high voltage, indicating 1.
- The state of the memory cell at the connection to the ground is defined during the chip fabrication.

Types of ROM

- The **programmable ROM** (PROM) is used when a few ROMs are required with a specific memory content. PROMs can be written only once using electric signals.
- The **erasable programmable ROM** (EPROM) can be read and written using electrical signals.
- Before a write operation is performed, the memory content of this memory chip is erased to retain its initial state by exposing the memory chip to the ultraviolet rays.
- The EPROM can be erased and updated repeatedly, and like ROM and PROM, it retains its memory content even if there is no power supply. Compared to ROM and PROM the EPROM is more expensive.

- The **electrically erasable programmable ROM** (EEPROM) is the ROM memory that can be erased selectively and written multiple times.
- Unlike the EPROM, where the entire data gets erased when exposed to the ultraviolet rays, in EEPROM, the selective data can be erased without removing the memory chip from the system, as with a higher-than-normal electrical voltage, the data can be erased.
- The EEPROM is a little complex when compared to the EPROM, as it requires a different electrical voltage to erase the data.